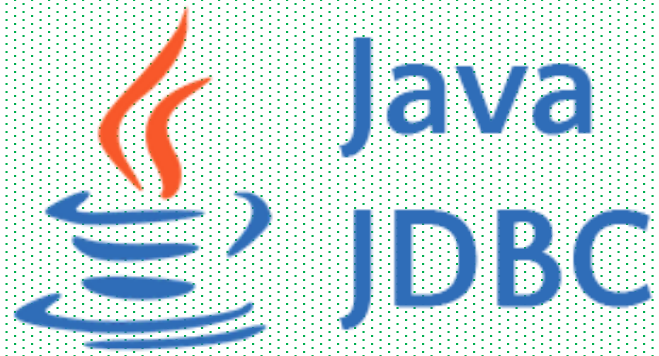
Persistencia en Java con JDBC



Persistencia con JDBC – Motivación

Escribir código ligado a una BBDD concreta trae problemas.

Acoplamiento

- El código queda ligado a un motor concreto *(MySQL, PostgreSQL, Oracle…).*
- Cambiar de BBDDs implica rehacer código.
- Dificulta la mantenibilidad del sistema.

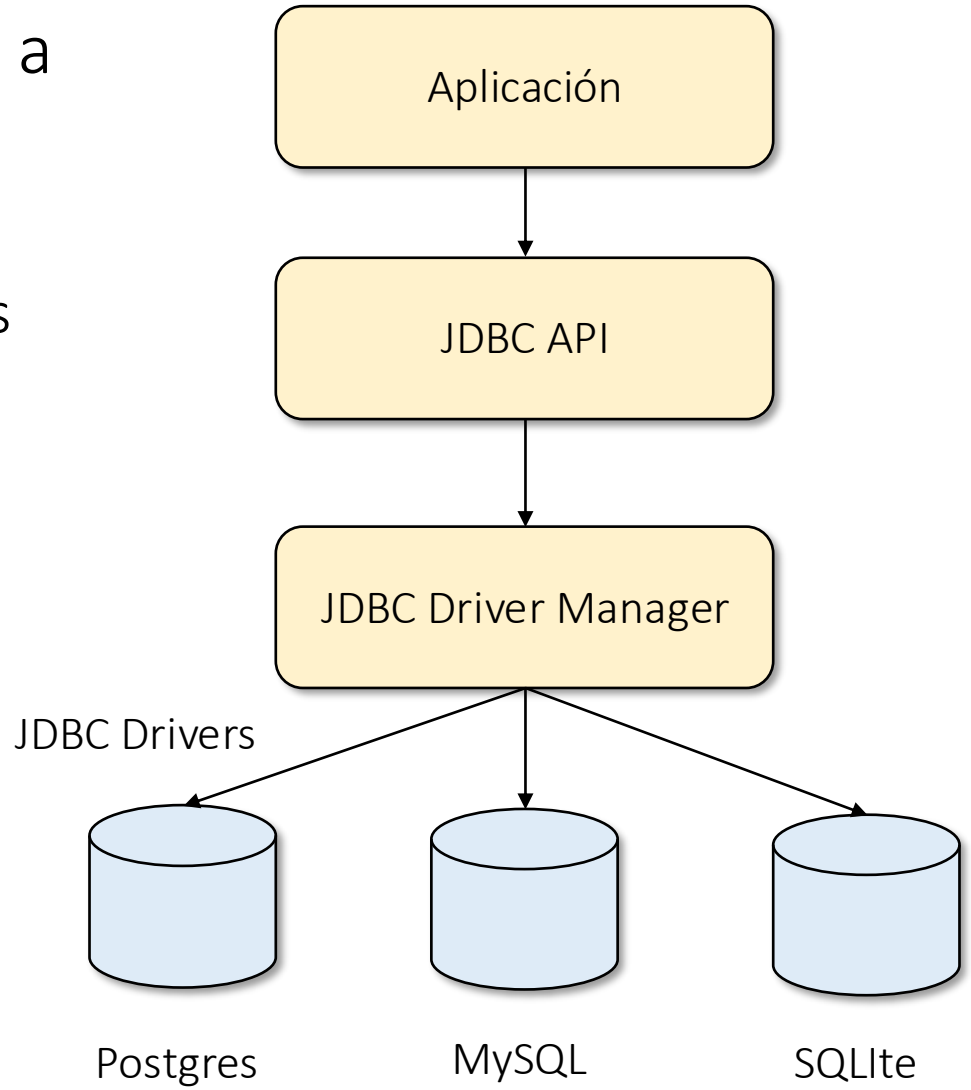
Portabilidad

- La aplicación no puede ejecutarse fácilmente en otros entornos.
- Las BBDDs ofrecen extensiones propias.
  Si no son estándar, tenemos menos portabilidad.

Persistencia con JDBC – Arquitectura

JDBC es la API estándar de Java para acceso a BBDD relacionales.

- **Capa de bajo nivel.**
  JDBC sólo proporciona interfaces y algunas clases básicas.
- **Cada BBDD proporciona su propio driver.**
  Los drivers proporcionan implementaciones concretas para cada motor de base de datos, lo que permite fácilmente pasar de una a otra.



Persistencia con JDBC – Clases principales

Clase	Descripción
`DriverManager`	Gestiona los drivers JDBC y establece conexiones con la BBDD.
`Connection`	Representa una sesión con una BBDD específica
`Statement`	Representa una consulta SQL estática (sin parámetros)
`PreparedStatement`	Representa una consulta SQL dinámica (con parámetros)
`CallableStatement`	Representa la invocación a un procedimiento almacenado
`ResultSet`	Representa el resultado de una consulta. Permite navegar las filas del resultado.
`SQLException`	Gestionar las excepciones de la BBDD.

Persistencia con JDBC – Configuración

- El driver tiene que estar accesible en `classpath`.
- En Maven:

¿Por qué? 

```
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.49.1.0</version>
    <scope>runtime</scope>
</dependency>
```




```
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.7.5</version>
    <scope>runtime</scope>
</dependency>
```



Persistencia con JDBC – Conexión

```
public class BaseDeDatos implements AutoCloseable {
  private static final String URL = "jdbc:sqlite:basedatos.db";
  private Connection conexion;

  public BaseDeDatos() throws SQLException {
    conexion = DriverManager.getConnection(URL);
  }

  @Override
  public void close() throws Exception {
    try {
      if (conexion != null) {
        conexion.close();
      }
    } catch (SQLException e) {
      e.printStackTrace();
    }
  }
}
```

Para indicar a qué BBDD conectar, usamos una JDBC URL.
Especifica el driver específico y la ubicación del recurso.

Sesiones

- `getConnection` nos da una sesión activa con nuestra BBDD.
- Las conexiones consumen recursos, por lo que debemos de cerrarlas.
- Tienen estados
  (ACTIVA, CERRADA, EN TRANSACCIÓN, …)

Excepciones

- Debemos de manejar excepciones, ya que estamos usando recursos externos.

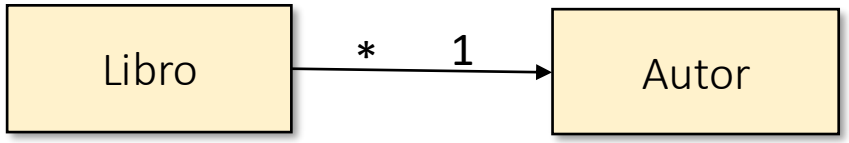
Persistencia con JDBC – Crear tablas

```
private void crearTablas() throws SQLException {
  String sqlAutores = """
    CREATE TABLE IF NOT EXISTS Autores (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      nombre TEXT NOT NULL,
    );""";

  String sqlLibros = """
    CREATE TABLE IF NOT EXISTS Libros (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      titulo TEXT NOT NULL,
      autor_id INTEGER,
      FOREIGN KEY (autor_id) REFERENCES Autores(id)
  );""";

  try (Statement stmt = conexion.createStatement()) {
    stmt.execute(sqlAutores);
    stmt.execute(sqlLibros);
  }
}
```

- El código SQL se construye como cadenas de texto.
- El programador debe conocer el modelo relacional.
- Las relaciones entre tablas las debemos de modelar nosotros mismos.
- JDBC permite enviar instrucciones SQL directamente a la base de datos mediante `statements`.



Persistencia con JDBC – Persistencia

```
public void addLibro(Libro libro) {
  String sql = "INSERT INTO Libros (titulo, autor_id) VALUES (?, ?)";
  try (PreparedStatement pstmt = conexion.prepareStatement (
    sql,
    Statement.RETURN_GENERATED_KEYS
  )) {
    pstmt.setString(1, libro.getTitulo());
    pstmt.setLong(2, libro.getAutor().getId());
    pstmt.executeUpdate();
    ResultSet rs = pstmt.getGeneratedKeys();
    if (rs.next()) {
      long codigo = rs.getLong(1);
      libro.setId(codigo);
    } else {
      throw new IllegalStateException();
    }
  } catch (SQLException e) {
    e.printStackTrace();
    throw new RuntimeException(e);
  }
}
```

prepareStatement

- SQL parametrizado.
- Evita concatenar cadenas. Más seguro. Previene inyección de SQL.
- Tenemos placeholders posicionales (?) para insertar los datos.

getGeneratedKeys

JDBC permite recuperar claves generadas.

Persistencia con JDBC – Transacciones

```
public static void main(String[] args) throws Exception {
  try (BaseDeDatos bd = new BaseDeDatos()) {
    Autor cervantes = new Autor("Miguel de Cervantes");
    bd.addLibro(new Libro("Don Quijote de la Mancha", cervantes));
    bd.addLibro(new Libro("Novelas ejemplares", cervantes));

    System.out.println("Autores:");
    bd.getAutores().forEach(a -> System.out.println("  " + a));

    System.out.println("Libros:");
    bd.getLibros().forEach(l -> System.out.println("  " + l));
  }
}
```

¿Qué problema hay?

Transacción

Secuencia de acciones que se ejecutan como una unidad (todas ok o ninguna)

AutoCommit

- Por defecto es *true*
- Cada sentencia SQL es una transacción

Gestión de transacciones

- conn.setAutoCommit(false)
- Hacer commit explícito (conn.commit())
- Si falla o se hace *rollback,* no se guarda nada

Persistencia con JDBC – Consultas

```
public List<Autor> getAutores() {
  String consulta = "SELECT id, nombre FROM Autores";

  try (Statement stmt = conexion.createStatement();
    ResultSet rs = stmt.executeQuery(consulta)) {

    List<Autor> autores = new ArrayList<>();
    while (rs.next()) {
       Autor autor = new Autor(
         rs.getLong("id"),
         rs.getString("nombre")
       );
       autores.add(autor);
    }

    return autores;
  } catch (SQLException e) {
    e.printStackTrace();
    throw new RuntimeException(e);
  }
}
```

ResultSet

- funciona como un cursor: apunta a una fila del resultado y avanza con `next`
- Permite leer columnas de la fila actual.
- Los datos no se cargan todos en objetos automáticamente.

Mapeo manual objeto–relacional

- Por cada fila de la tabla, se crea un objeto

Persistencia con JDBC – Ventajas y desventajas

Ventajas

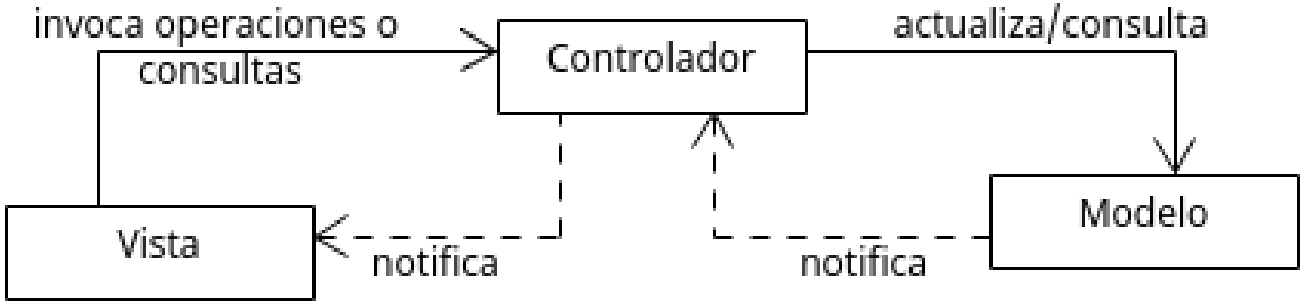
- Sencillo de usar.
- Control sobre cómo se realiza la persistencia.
- Ejecución más rápida de consultas simples (no siempre hace falta el mapeo).

Desventajas

- Creación de gran cantidad de código manual.
- Riesgo de errores.
- No hay gestión automática de transacciones.
- No hay *lazy loading.*

Patrón Repositorio – Motivación

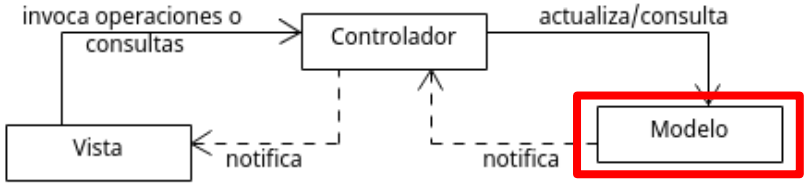
Quiero persistir los datos de mi modelo ¿Dónde lo implemento?




```
Connection conn = DriverManager.getConnection("jdbc:mysql:...");
PreparedStatement stmt = conn.prepareStatement("INSERT INTO Producto VALUES(...");
stmt.setString(1, codigo);
...
stmt.executeUpdate();
stmt.close();
conn.close();
```

Patrón Repositorio – Motivación

Quiero persistir los datos de mi modelo ¿Dónde lo implemento?



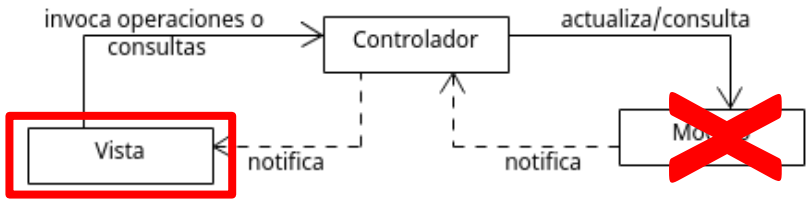

```
public Producto(String codigo, String descripcion,...) {
    this.codigo = codigo;
    this.descripcion = descripcion;
    ...
    PreparedStatement stmt =
        conn.prepareStatement("INSERT INTO Producto ..."));
    ...
}
```

Problemas si lo hago en el **modelo**:

- Mezcla de responsabilidades: lógica de negocio y persistencia
- Puede producirse código duplicado
- Alto coste cambio de persistencia
- Difícil testear componentes por separado

Patrón Repositorio – Motivación

Quiero persistir los datos de mi modelo ¿Dónde lo implemento?



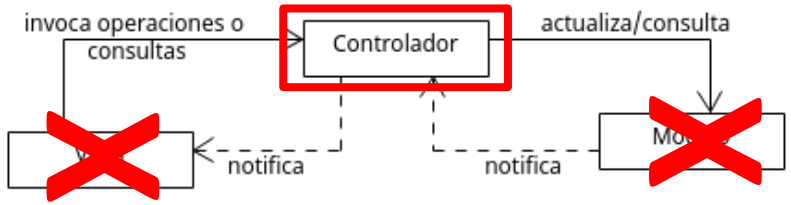

```
crearBtn.setOnAction(e -> {
    String codigo = codigoTF.getText();
    String descripcion = descripcionTF.getText();
        ...
    PreparedStatement stmt =
        conn.prepareStatement("INSERT INTO Producto ..."));
        ...
});
```

Problemas si lo hago en la **vista**:

- Mezcla de responsabilidades: vista y persistencia
- Puede producirse código duplicado
- Alto coste cambio de persistencia
- Un cambio de interfaz (es común) me puede romper la persistencia
- Difícil testear componentes por separado

Patrón Repositorio – Motivación

Quiero persistir los datos de mi modelo ¿Dónde lo implemento?




```
public void creaProducto(String codigo, String descrip ... {

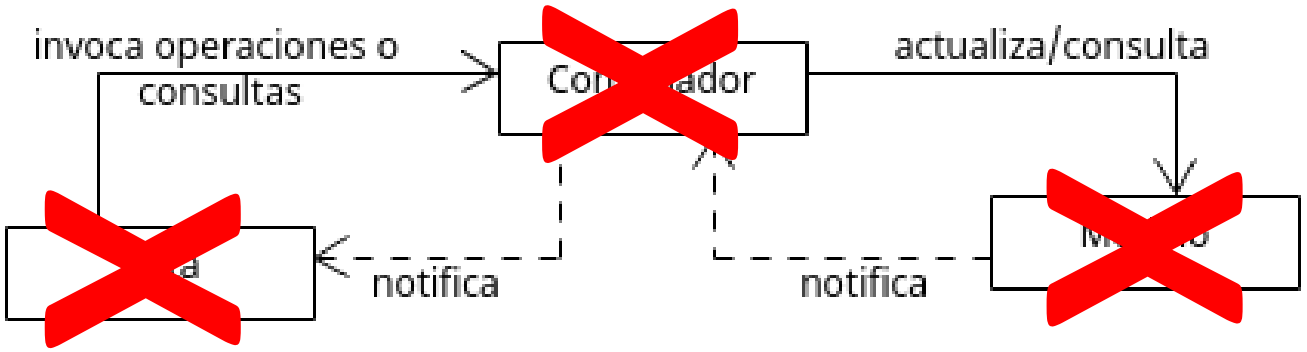
    Producto p = new Producto(codigo, descripcion, ...);
    ...
    PreparedStatement stmt =
       conn.prepareStatement("INSERT INTO Producto ..."));
    ...
}
```

Problemas si lo hago en la **controlador**:

- Mezcla de responsabilidades: coordinación y persistencia
- Un cambio de persistencia implica cambiar todo el controlador
- Difícil testear controlador/persistencia de forma aislada

Patrón Repositorio – Motivación

Quiero persistir los datos de mi modelo ¿Dónde lo implemento?

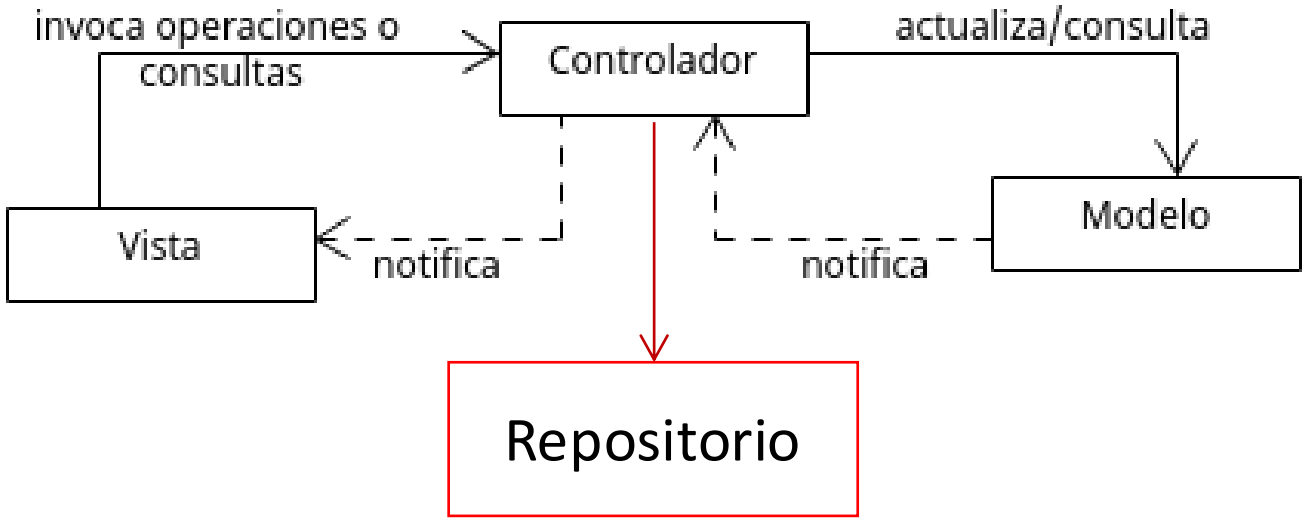


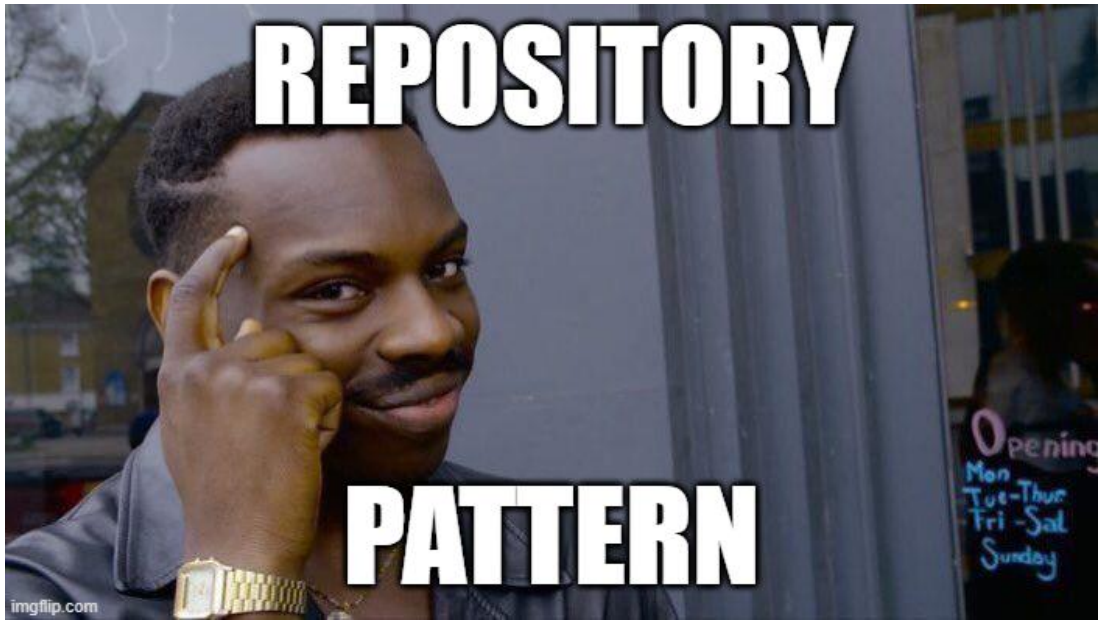


Fuente: https://imgflip.com/memegenerator

Patrón Repositorio – Motivación

Quiero persistir los datos de mi modelo ¿Dónde lo implemento?

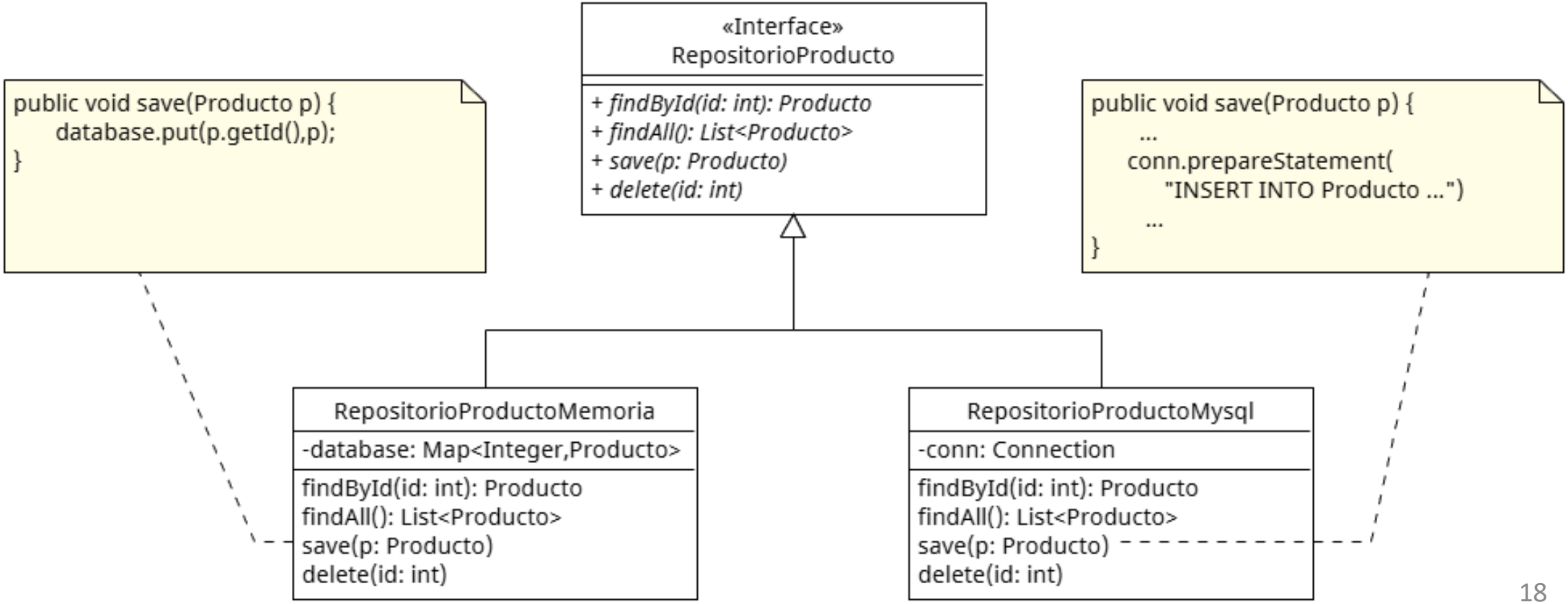




Fuente: https://imgflip.com/memegenerator

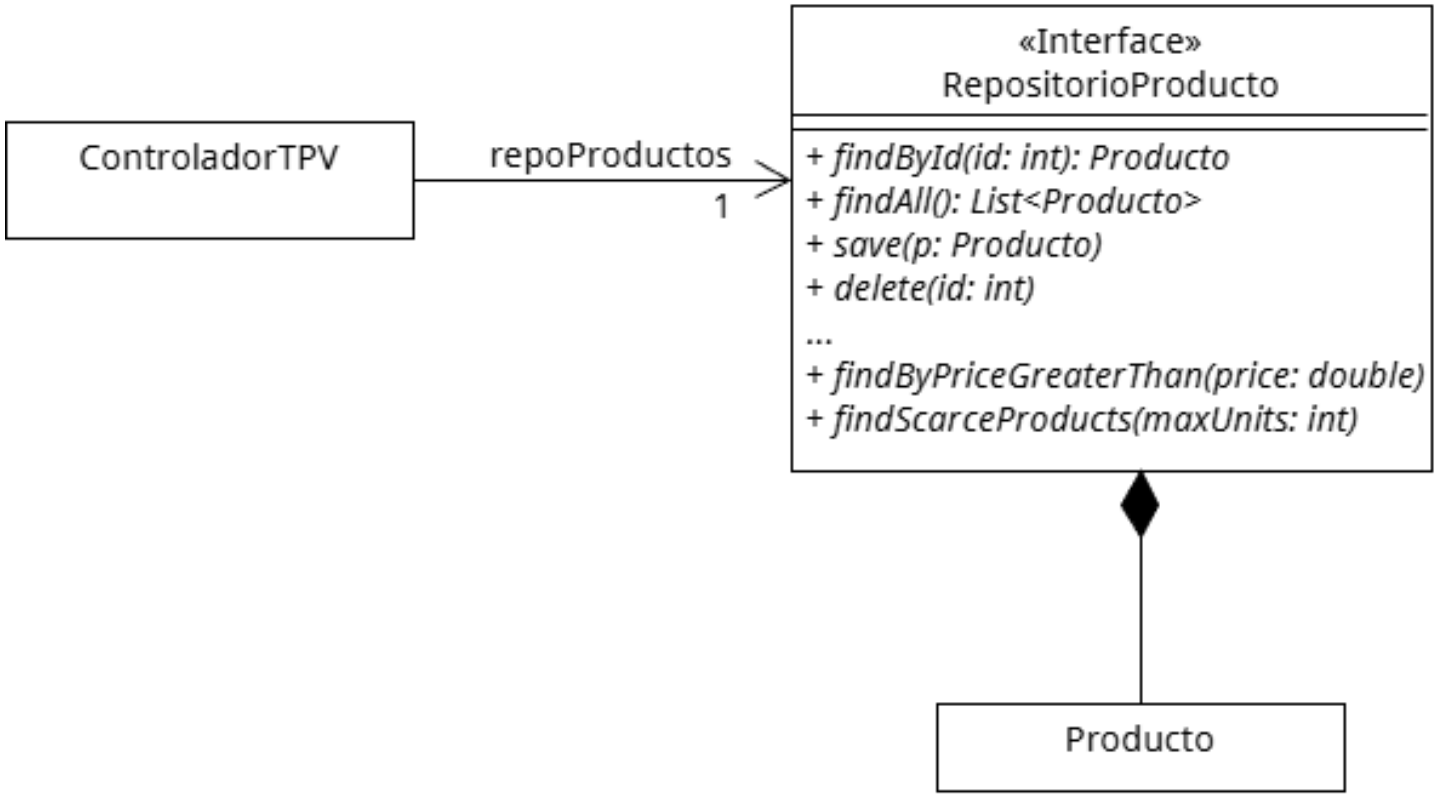
Patrón Repositorio – Implementación

- Componentes
  - **Interfaz del repositorio**: Define operaciones (CRUD, etc.)
  - **Implementación**: Acceso a datos basado en una tecnología concreta



Patrón Repositorio – Implementación

- Uso:
  - El controlador (sólo el controlador) conoce la **interfaz del repositorio**
  - La **implementación** concreta se inyectará al arrancar la aplicación

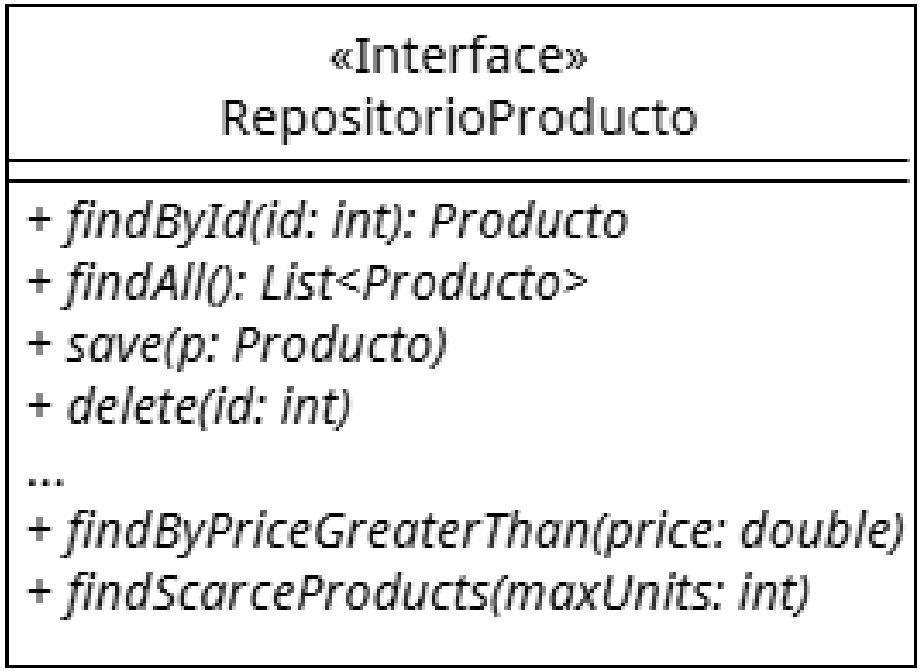


Patrón Repositorio – Resumen

- Consigue abstraer el acceso a datos

- Recomendaciones:
  - Evitar lógica compleja relativa al negocio
  - Incluir métodos importantes para el negocio (no solo CRUD)

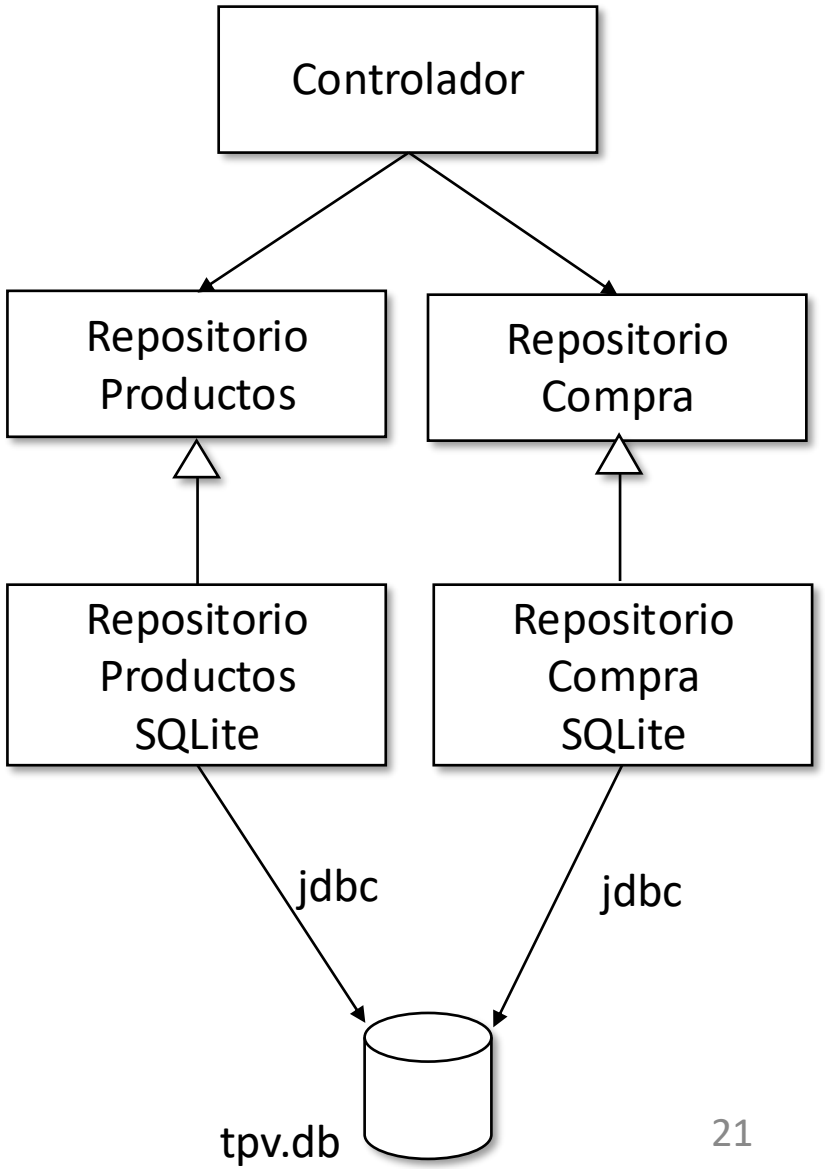
- Beneficios
  - Bajo acoplamiento
  - Testeo más sencillo
  - Reutilización
  - Escalabilidad/Evolución del Sistema



- No solo para MVC → Usado en arquitecturas de capas en general (arquitectura limpia, arquitectura hexagonal, etc.)

Persistencia en una arquitectura MVC

- Una implementación concreta de cada repositorio.
- Se elige la tecnología de base de datos
  - SQLite, H2, Postgres, etc.
- Los repositorios comparten la misma base de datos física



Persistencia en una arquitectura MVC

```
public class ProductoRepositorySQLiteImpl implements ProductoRepository {

 private final BaseDatosSQLite baseDatos;

 // Funciona como un catálogo, mantiene los datos en memoria y simplemente sincroniza
 // con la base de datos física. La lista se crea al construir el repositorio.
 private List<ProductoImpl> productos;

 // La gestión de la base de datos encapsulada en BaseDatosSQLite
 public ProductoRepositorySQLiteImpl(BaseDatosSQLite baseDatos) {
   this.baseDatos = baseDatos;
   try {
     this.productos = cargarProductos();
   } catch (SQLException e) {
     throw new RuntimeException(e);
   }

 }
```

…

Persistencia en una arquitectura MVC

```
private List<ProductoImpl> cargarProductos() throws SQLException {
  try (Connection conexion = baseDatos.abrirConexion()) {
     List<ProductoImpl> resultado = new ArrayList<>();
     String consulta = "SELECT codigo, descripcion, cantidad, " +
        "precio FROM producto ORDER BY codigo";
     try (Statement sentencia = conexion.createStatement();
       ResultSet filas = sentencia.executeQuery(consulta)) {
       while (filas.next()) {
         String codigo = filas.getString("codigo");
         String descripcion = filas.getString("descripcion");
         int cantidad = filas.getInt("cantidad");
         double precio = filas.getDouble("precio");

         resultado.add(new ProductoImpl(codigo, descripcion, cantidad, precio));
       }
     }
  }
  return resultado;
}
```

Persistencia en una arquitectura MVC

```
@Override
public Producto findById(String codigo) throws ElementoNoEncontradoException {
   return buscar(codigo).orElseThrow(() ->
      new ElementoNoEncontradoException("El producto " + codigo + " no existe"));
}


private Optional<ProductoImpl> buscar(String codigo) {
  return productos.stream().
      filter(prod -> prod.getCodigo().equals(codigo)).
      findFirst();
}
```

Las consultas se hacen con streams puesto que los datos están en memoria.
¿Posibles problemas?

Persistencia en una arquitectura MVC

```
@Override
public void addCompra(CarritoCompra compra) {
  // Si la compra ya existe no puedo insertarla
  if (buscar(compra.getIdCompra()).isPresent()) {
    throw new ElementoExistenteException("Ya existe " + compra.getIdCompra());
  }

  try (Connection conexion = baseDatos.abrirConexion();
     PreparedStatement insercion = conexion.prepareStatement(
         "INSERT INTO compra (id_compra) VALUES (?)")) {
     conexion.setAutoCommit(false);
     insercion.setString(1, compra.getIdCompra());
     insercion.executeUpdate();
     insertarItems(conexion, compra);
     conexion.commit();
  } catch (SQLException e) {
     throw new ErrorPersistenciaException(e);
  }
  compras.add(compra);
}
```

Una conexión por interacción

Gestionar transacción

Sincronizar el catálogo